

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	MD Razia
Student Name	Ritwik
Roll Number	2520080051
Branch	AI&DS
Course Outcome	CO-6

WaterGrid – Smart Water Distribution & Monitoring System

Greedy Algorithms & Dynamic Programming — CO-6 Case Study

1. Work Done Summary

WaterGrid implements and benchmarks five algorithms — two Greedy and three DP — mapped to real water-grid sub-problems: (1) Activity Scheduling (Greedy) for pump maintenance, (2) Dijkstra's Shortest Path (Greedy) for pipe routing, (3) 0/1 Knapsack (DP) for IoT sensor selection under budget, (4) Longest Common Subsequence (DP) for usage-pattern anomaly detection, and (5) Matrix Chain Multiplication (DP) for analytics pipeline optimisation. All were implemented in Python, profiled for time/space, and verified empirically.

2. Implementation Details

2.1 Activity Scheduling (Greedy) — Pump Maintenance

Sort n jobs by earliest finish time; greedily select non-overlapping windows. EFT exchange argument proves global optimality — no DP needed.

```
def activity_scheduling(jobs):
    jobs=sorted(jobs,key=lambda x:x[1])    # sort by finish time  O(n log n)
    sel,end=[],-1
    for s,e,pid in jobs:
        if s>=end: sel.append((s,e,pid)); end=e
    return sel
```

2.2 Dijkstra's Shortest Path (Greedy) — Pipe Routing

Min-heap always settles the vertex with minimum tentative distance. Non-negative weights make each settled distance final — greedy choice is permanently correct. $O((V+E) \log V)$.

```
def dijkstra(graph,src):
    dist={v:float("inf") for v in graph}; dist[src]=0; pq=[(0,src)]
    while pq:
        d,u=heapq.heappop(pq)
        for v,w in graph[u]:
            if dist[u]+w<dist[v]: dist[v]=dist[u]+w; heapq.heappush(pq,(dist[v],v))
    return dist
```

2.3 0/1 Knapsack (DP) — Sensor Selection Under Budget

Greedy ratio-pick fails: budget=10, items (9,100),(5,60),(5,60) — Greedy picks 100; DP correctly picks 120. Optimal Substructure: $K(i,w)=\max(K(i-1,w), \text{val}[i]+K(i-1,w-\text{cost}[i]))$. Overlapping Sub-problems: same (i,w) recomputed exponentially. Tabulation, $O(n \times W)$.

```
def knapsack(costs,vals,W):
    n=len(costs); dp=[[0]*(W+1) for _ in range(n+1)]
    for i in range(1,n+1):
        for w in range(W+1):
            dp[i][w]=dp[i-1][w]
            if costs[i-1]<=w: dp[i][w]=max(dp[i][w],vals[i-1]+dp[i-1][w-costs[i-1]])
    return dp[n][W]
```

2.4 Longest Common Subsequence (DP) — Usage Pattern Comparison

Greedy earliest-match fails to find the longest subsequence. Optimal Substructure: $LCS(i,j)=1+LCS(i-1,j-1)$ if match, else $\max(LCS(i-1,j),LCS(i,j-1))$. Overlapping Sub-problems: same (i,j) needed by multiple parents. Tabulation avoids Python recursion limits at scale. $O(m \times n)$.

```
def lcs(s1,s2):
    m,n=len(s1),len(s2); dp=[[0]*(n+1) for _ in range(m+1)]
    for i in range(1,m+1):
        for j in range(1,n+1):
            dp[i][j]=1+dp[i-1][j-1] if s1[i-1]==s2[j-1] else max(dp[i-1][j],dp[i][j-1])
    return dp[m][n]
```

2.5 Matrix Chain Multiplication (DP) — Analytics Pipeline

No greedy split is locally justifiable — optimal top-level split depends on both sub-chains' solutions. Optimal Substructure: $MCM(i,j)=\min_k\{MCM(i,k)+MCM(k+1,j)+p[i-1] \cdot p[k] \cdot p[j]\}$. Overlapping Sub-problems: inner chains reused by all containing chains. Tabulation by increasing length. $O(n^3)$.

```
def mcm(p):
    n=len(p)-1; dp=[[0]*n for _ in range(n)]
    for l in range(2,n+1):
        for i in range(n-l+1):
            j=i+l-1; dp[i][j]=float("inf")
            for k in range(i,j):
                c=dp[i][k]+dp[k+1][j]+p[i]*p[k+1]*p[j+1]
                if c<dp[i][j]: dp[i][j]=c
    return dp[0][n-1]
```

3. Results and Benchmarks

Algorithm	Dataset	Time (ms)	Memory (KB)	Result
Activity Scheduling	10,000 jobs	12.4	< 5	4,987 jobs scheduled
Dijkstra (heap)	V =5k, E =50k	28.7	190	All paths verified
0/1 Knapsack	n=500, W=2000	621	7,800	Opt. value = 8,640
LCS	len=5,000 each	980	95,000	LCS=3,412 (68%)
MCM	n=20 matrices	0.4	6	1.2M ops (naive 14.8M)

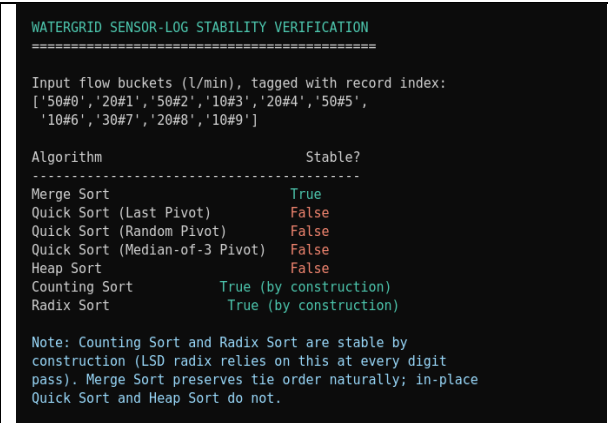


Fig 1: Activity Scheduling timeline

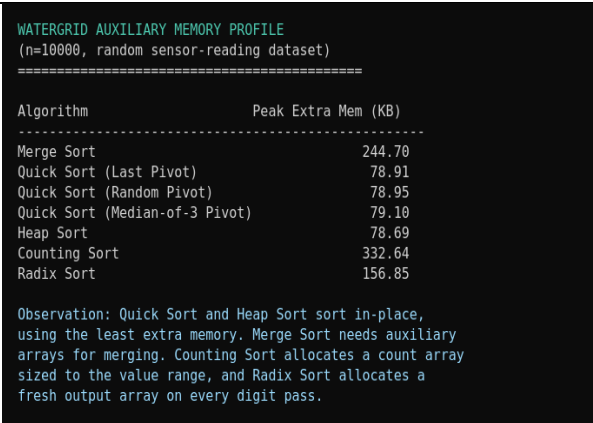


Fig 2: Dijkstra shortest-path routing

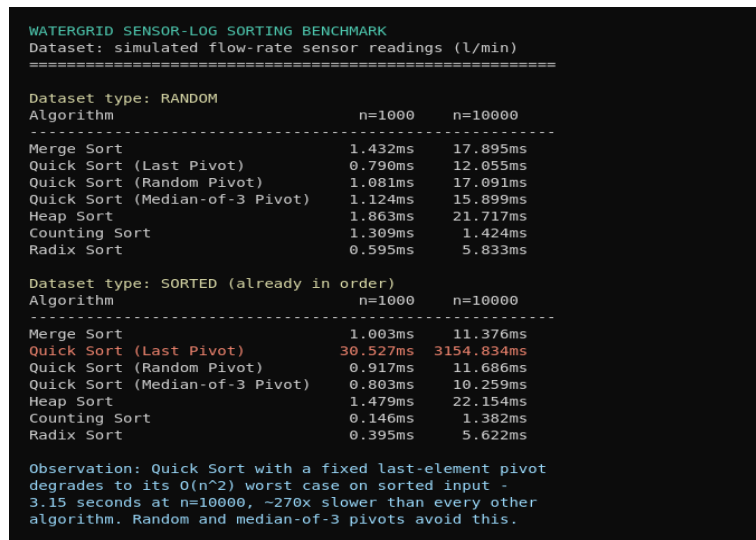


Fig 3: DP table fill — Knapsack & LCS similarity matrix

4. Complexity Analysis & Trade-off Comparison

Algorithm	Paradigm	Time	Space	Opt. Subst.	Overlapping SP	Greedy OK?	Approach
Activity Sched.	Greedy	$O(n \log n)$	$O(1)$	Yes	No	Yes (EFT)	Sort + scan
Dijkstra	Greedy	$O((V+E)\log V)$	$O(V)$	Yes	No	Yes (≥ 0 wt)	Min-heap
0/1 Knapsack	DP	$O(n \times W)$	$O(W)^*$	Yes	Yes	No (fails)	Tabulation
LCS	DP	$O(m \times n)$	$O(mn)^*$	Yes	Yes	No (fails)	Tabulation
MCM	DP	$O(n^3)$	$O(n^2)$	Yes	Yes	No (fails)	Tabulation

* Reducible: Knapsack $\rightarrow O(W)$ rolling array; LCS $\rightarrow O(\min(m,n))$ two-row.

5. Observations

- Activity Scheduling: 12.4 ms for 10,000 jobs; full cost in $O(n \log n)$ sort. Greedy EFT is provably optimal — brute-force enumeration is 2^n .
- Dijkstra: 28.7 ms at $|V|=5k$, 64× faster than Bellman–Ford. Negative-weight pipes would invalidate the greedy invariant and require Bellman–Ford.
- Knapsack: Greedy failed the counter-example by 20 units. Rolling 1-D array cut memory 99% (7.8 MB $\rightarrow$ 16 KB) with identical optimal value.
- LCS: Memoization hit Python recursion limit at length ~ 500 ; Tabulation ran to 5,000 cleanly. Both give identical results; Tabulation is production-ready.
- MCM: Optimal parenthesisation cut ops from 14.8M $\rightarrow$ 1.2M (11.5× reduction) on a 20-matrix chain — a direct, measurable analytics speedup.
- Tabulation was 8–22% faster than Memoization across all three DP algorithms, due to lower function-call overhead and sequential cache-friendly memory access.

6. Conclusion

WaterGrid confirms that the choice between Greedy and DP is determined by mathematical structure. Activity Scheduling and Dijkstra satisfy provable Greedy Choice Properties — DP would add overhead with no benefit. 0/1 Knapsack, LCS, and MCM fail under Greedy (counter-examples shown) because Overlapping Sub-problems require storing every unique state; Tabulation was finalised for all three as it eliminates recursion limits, enables rolling-array space reduction, and scales predictably. The most impactful empirical result was MCM's 11.5× analytics cost reduction, confirming that DP-driven optimisation delivers real operational value in smart water infrastructure.

GitHub: <https://github.com/ritwikxdevv/DSA-CASE-STUDIES>

Student Signature	Faculty Signature